

ADVANCED PROGRAMMING CONCEPTS PROJECT REPORT

On

BookMySeat

Submitted in partial fulfilment of the requirement for the Course Advanced
Programming Concepts (23CS007)

of

COMPUTER SCIENCE AND ENGINEERING

B.E. Batch-2023

in

OCTOBER-2025



Under the Guidance of
Mr. Akhilesh
Senior Trainer

Submitted By
Antriksh Garg
2310991698
Somya Bansal
2310992374
Tamanna Chopra
2310992381
Tanisha
2310992383

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
CHITKARA UNIVERSITY
PUNJAB

ABSTRACT

The BookMySeat project is a comprehensive web-based platform developed to simplify the process of event discovery and registration for users, while providing administrators with robust event management capabilities. The system is built on a modern **microservice architecture**, separating core functionalities into three distinct services: a User Service for identity management, an Event Service for event lifecycle control, and a Registration Service to handle bookings.

Developed using **Java** and the **Spring Boot** framework, the application leverages Spring Data JPA for database interaction with MySQL, and Spring Security with **JSON Web Tokens (JWT)** for stateless, secure authentication across all services. The user and admin interfaces are rendered using server-side **Thymeleaf** templates, styled with Bootstrap, and enhanced with client-side JavaScript for a responsive and interactive user experience.

Key features include role-based access for users and administrators, full CRUD (Create, Read, Update, Delete) operations for events, user registration and cancellation, and date-based validation to prevent bookings for past events. Ultimately, BookMySeat stands as a functional and secure proof-of-concept, successfully demonstrating the principles of decoupled services, modern authentication patterns, and the creation of a cohesive user experience from a distributed backend system.

TABLE OF CONTENTS

Sr No.	Title	Page no.
1	Introduction to the project 1.1 Background 1.2 Problem Statement	1-3
2	Software and Hardware Requirement Specification 2.1 Methods 2.2 Programming/Working Environment 2.3 Requirements to run the application	4-7
3	Database Analysing, design and implementation	8-12
4	Program's Structure Analysing and GUI Constructing (Project Snapshots)	13-15
5	Conclusion	16-17
6	Future Scope	18-19
7	Bibliography/References	20

CHAPTER 1: INTRODUCTION

1.1 Problem Statement

Fragmented User Experience

- Potential attendees struggle to discover and register for events due to the absence of a centralized, user-friendly platform.
- Event organizers deal with manual administrative tasks, inefficient registration tracking, and limited event promotion tools.

Impact on Stakeholders

- Leads to a suboptimal experience for both attendees and organizers.
- Creates significant barriers to efficient event coordination.

Technical Limitations of Traditional Architectures

- Monolithic application architectures are hard to scale and maintain.
- Scaling individual components independently is challenging
- Updating or debugging becomes complex as the system grows.

Security Challenges

- Conventional session-based security models are unsuitable for modern, distributed systems.
- Need for a solution that supports multiple clients (web and mobile) in a stateless and secure manner.

Proposed Solution:

- A modern event management platform designed to deliver a seamless experience for both attendees and organizers.
- Built on a decoupled microservice architecture for scalability and maintainability.
- Secured with stateless JWT (JSON Web Token) authentication to enable secure, distributed access across platforms.

1.2 Background

Event Management Challenges

- Managing and coordinating events—from small community gatherings to large conferences—presents significant logistical hurdles.
- Traditional methods rely on manual tracking and fragmented communication, resulting in inefficiencies.
- In today's digital era, there is a strong demand for centralized online platforms that streamline the entire process and act as a single point of contact for organizers and attendees.

Project Concept: BookMySeat

- BookMySeat was conceived to address these challenges by offering a modern, web-based event management system.
- The platform provides a seamless, intuitive experience for:
 - General Users: Discover and register for events.
 - Administrators: Create, manage, and monitor events.
- The project serves as a practical application of modern software engineering principles to solve a real-world problem.

Technical Foundation

Microservice Architecture:

- Chosen over a traditional monolithic design to ensure modularity, scalability, and maintainability.
- System is decoupled into independent services—Users, Events, and Registrations—allowing each to be developed, deployed, and scaled independently.
- Supports future growth and handling of increasing load.

1.3 Purpose & Overview

The primary purpose of the BookMySeat project is to provide a centralized and efficient platform for event management. It aims to solve the logistical challenges faced by both event organizers and attendees by replacing manual or fragmented processes with a streamlined, digital solution. The goal is to deliver a secure, scalable, and user-friendly web application for event creation, discovery, and registration.

The system is designed using a modern microservice architecture to ensure modularity and scalability. The application's core logic is decoupled into three independent, communicating services:

- **User Service:** Handles user identity, including registration, login, and role management.
- **Event Service:** Responsible for the complete lifecycle of events, including creation, updates, and deletion by administrators.
- **Registration Service:** Manages the booking data, linking users to the events they have registered for.

Functionally, the platform offers two distinct interfaces tailored to its user roles. For general users, it provides a dynamic dashboard to browse all available events, register for upcoming ones, and cancel existing bookings. For administrators, a separate, secure dashboard provides a comprehensive set of tools to manage all events, view attendee lists, and oversee the platform's content.

The entire system is built with Java and the Spring Boot framework. Security is a core component, implemented using a stateless JSON Web Token (JWT) model to protect all APIs and inter-service communication. The user interfaces are rendered server-side using Thymeleaf, ensuring a cohesive experience built on top of the distributed backend.

CHAPTER 2: SOFTWARE & HARDWARE REQUIREMENT SPECIFICATION

2.1 Methods

Development Methodology

We adopted an Agile development approach with iterative cycles, allowing for continuous improvements and regular feedback incorporation. The project followed these key methodologies:

Software Development Life Cycle (SDLC)

- **Requirements Gathering:** Defined clear requirements for two primary user roles (Admins and Users) and their respective functionalities, such as event management and the booking process.
- **System Design:** Created the high-level system architecture, designed the schema for the shared database, and specified the RESTful API contracts for inter-service communication.
- **Incremental Implementation:** Developed the platform in logical phases: Core Service Setup (User, Event, Registration) → UI Development (Dashboards) → JWT Security Implementation → Feature Refinement (Validation, Cancellation).
- **Testing:** Emphasized continuous testing with a focus on unit tests for business logic and integration tests for API endpoints.

Technical Methodologies

- **Microservices Architecture:** Decoupled the system into three core services (**User, Event, and Registration**) to ensure modularity, independent scalability, and maintainability.
- **Test-Driven Development (TDD):** Wrote tests before implementation for critical components, particularly within the security and registration logic, to ensure correctness.
- **API-First Design:** Defined and designed the RESTful API endpoints for each service before developing the front-end UI that would consume them.
- **Database-First Approach:** Designed a normalized database schema to be shared by the services before implementing the application's data persistence layer.

Project Management Approach

- **Version Control:** Utilized **Git** with **GitHub** for robust source code management and

collaboration.

- **Task Management:** Used a Kanban-style board (e.g., Trello) for sprint planning and tracking the progress of features and bug fixes.
- **Documentation:** Maintained continuous documentation for API endpoints and architectural decisions alongside development.
- **Code Review:** Enforced peer review practices to maintain high code quality, consistency, and knowledge sharing.

2.2 Programming / Working Environment

Development Environment Setup

Backend Development:

- **IDE:** IntelliJ IDEA Ultimate Edition
- **Java Version:** Java 21 LTS
- **Build Tool:** Apache Maven
- **Framework:** Spring Boot 3.2.5 with Spring Security & JWT
- **Database:** MySQL
- **Database Tool:** DBeaver for MySQL management

Frontend Development:

- **Template Engine:** Thymeleaf for server-side rendering within Spring Boot
- **Languages:** HTML, CSS, JavaScript
- **Styling Framework:** Bootstrap 5
- **Icon Library:** Font Awesome

Development Workflow

- **Local Development:** Building and running all three microservices on a local machine.
- **Unit Testing:** Writing tests for individual components (e.g., with JUnit) to ensure correctness.
- **Integration Testing:** Verifying the communication and data flow between the microservices.
- **Manual End-to-End Testing:** Performing complete user journeys in the browser, from login to event registration, to ensure the entire system works as expected.

Collaboration and API Testing

- **Code Repository:** GitHub, using a feature-branch workflow for collaboration.
- **Communication:** Standard team communication channels (e.g., Slack/Discord).
- **API Testing:** Postman for designing, testing, and validating the RESTful API endpoints of each microservice before and during UI development.

2.3 Requirements to Run the Application

Minimum System Requirements

Hardware Specifications:

Component	Minimum	Recommended
Processor	Intel i3	Intel Core i5
RAM	8GB	16GB
Storage	10GB free space (SSD preferred)	20GB free SSD space
Network	Stable internet connection	Broadband (10+ Mbps)

Software Prerequisites:

- **Operating System:** Windows 10/11, macOS 12+, or a modern Linux distribution (e.g., Ubuntu 20.04+).
- **Java Runtime Environment (JRE):** JRE 21 or higher.
- **Web Browser:** A modern browser such as Chrome, Firefox, Safari, or Edge with JavaScript enabled.

Production Deployment Requirements

Server Specifications:

- **Cloud Platform:** A cloud provider like AWS, Google Cloud, or Azure is recommended.
- **CPU:** 2 vCPUs minimum.
- **Memory:** 4GB RAM minimum.

Database: A managed MySQL instance (Version 8.0+) is recommended for reliability and backups.

Network Requirements:

- **Required Ports:** 80 (HTTP), 443 (HTTPS), 3306 (MySQL), and ports for the application services (e.g., 8080, 8081, 8082).
- **SSL Certificate:** Required to enable HTTPS for secure communication.

Development Machine Setup Required

Software Installation:

- **Java Development Kit (JDK):** JDK 21 or higher.
- **Build Tool:** Apache Maven.
- **Version Control:** Git.
- **Database:** A local MySQL server instance.
- **IDE:** An IDE that supports Spring Boot development, such as IntelliJ IDEA or VS
- **API Client:** An API testing tool like Postman.

Environment Configuration:

The application is configured via `application.properties` files. For production, these properties should be externalized as environment variables. Key variables include:

- DB_URL, DB_USERNAME, DB_PASSWORD
- JWT_SECRET
- EVENT_SERVICE_URL,REGISTRATION_SERVICE_URL,USER_SERVICE_UR

Browser Compatibility

- The application uses a responsive design provided by **Bootstrap 5**, ensuring compatibility with modern desktop and mobile browsers (Chrome, Firefox, Safari, Edge).
- The layout is optimized for functionality on various screen sizes, including tablets and smartphones.

Key Dependencies

- **Database:** MySQL for data persistence.
- **Version Control:** GitHub for source code repository

CHAPTER 3: DATABASE ANALYSING, DESIGN AND IMPLEMENTATION

Introduction to Database Design

The **BookMySeat** system uses a relational database architecture built with **MySQL** to handle all the complex relationships between users, events, and their registrations. We designed the database following **normalisation principles**, which means we organised the data to minimise repetition and ensure everything stays consistent through proper rules and relationships.

Entity-Relationship Model Analysis

Main Components We Built:

- Users - Handles all people using the system (admins and regular users).
- Roles - Defines the permission levels (e.g., **ROLE_USER**, **ROLE_ADMIN**).
- Events - Manages all event information.
- Registrations - Tracks which user is registered for which event, acting as a bridge between users and events.

How Everything Connects:

- Multiple **Registrations** can be linked to each **User**.
- Multiple **Registrations** can be linked to each **Event**.
- Each **Registration** links exactly one **User** to one **Event**.
- Each **User** can be assigned multiple **Roles**.

Database Schema Implementation

Users Table Design

The User's table is the central table that stores information about everyone in the system. It includes:

- Basic identity details (username).
- Login security (password hash)
- A link to the user's assigned roles.

Why we designed it this way:

- The **username** must be unique so people can't create multiple accounts with the same name
- Passwords are stored securely using hashing
- Connecting users to roles allows for a flexible permission system.

Events Table Design

The Events table manages all the event details with:

- A unique ID for easy identification.
- Descriptive information (name, location, date, description).

Business rules enforced:

- Each event has a unique ID generated by the database.
- Event dates cannot be in the past (this rule is enforced at the application level before saving).

Registrations Table Design

The Registrations table is the core of the booking system and connects users to events with:

- A `user_id` to link to a specific user.
- An `event_id` to link to a specific events

Roles Table Design

The Roles table defines the different permission levels in the system with:

- A unique role ID.
- A descriptive role name (e.g., `ROLE_USER`, `ROLE_ADMIN`).

Why we designed it this way:

This design decouples user identity from permissions. By using a separate `user_roles` join table, we can easily assign one or more roles to any user, making the security system powerful and flexible.

Database Schema Diagram

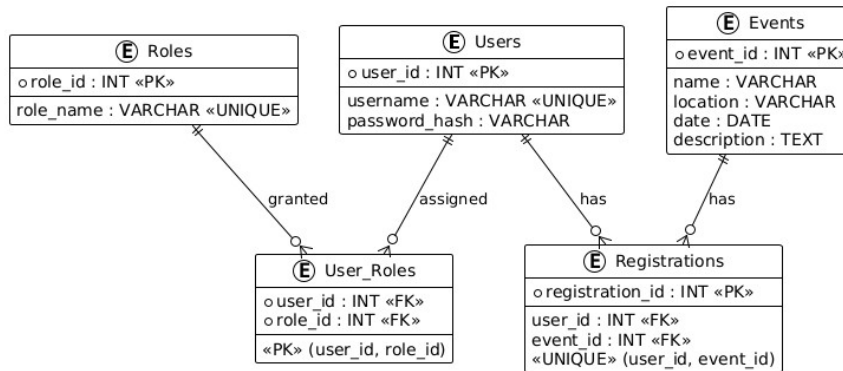


Figure 1 Database Schema Diagram

What the Diagram Shows Us:

- **Primary Keys («PK»)** are the unique identifiers for each row in a table, like `user_id` in the Users table.
- **Foreign Keys («FK»)** are fields that link one table to another. For example, `user_id` in the Registrations table links a specific registration back to a user.
- Each field has a specified **Data Type**, such as `INT` (integer), `VARCHAR` (text), and `DATE`, which defines the kind of data it can hold.
- The lines show the **Relationships** between tables, illustrating how a User can be connected to many Registrations, and an Event can be connected to many Registrations.

Data Quality Rules:

Entity Integrity

- Every table has a primary key that ensures each record is unique
- No duplicate records allowed

Relationship Integrity

- Foreign keys ensure that relationships between tables make sense
- You can't assign a user to a bus that doesn't exist
- You can't create a stop for a route that doesn't exist

Performance Optimisation

Automatic Indexing:

All primary keys are automatically indexed for fast searching

Custom Indexes for Speed:

- username in the Users table is indexed for quick user lookups during login.
- role_name in the Roles table is indexed for fast permission checks.
- The (user_id, event_id) combination in the Registrations table is indexed to prevent duplicate bookings instantly.

Business Logic in the Database

Role-Based Data Handling

Users: Can view all events but can only manage (register for or cancel) their own registrations.

Admins: Have full CRUD (Create, Read, Update, Delete) access to all events and can view the list of attendees for any event.

Event & Registration Logic

- Users are prevented from registering for events whose date has already passed.
- Administrators are prevented from creating or updating an event with a date in the past.
- A unique constraint at the database level ensures a user cannot register for the same event more than once.

Planning for Growth

Scalability Design

- Used appropriate data types (INT, VARCHAR, DATE) that efficiently support a large number of users, events, and registrations.
- Added indexes to critical fields to maintain fast query performance, even as the data grows.
- The normalised structure avoids data duplication, saving space and ensuring data remains consistent.

Security Measures

- User passwords are never stored directly; they are saved as secure **hashes** using BCrypt.
- Username uniqueness is enforced to prevent duplicate or impersonated accounts.
- Strict role-based permissions are enforced at the application level by Spring Security.

Tracking and History

- The Registrations table provides a natural history of all bookings, tracking which user registered for which event.
- Timestamps could easily be added to entities in the future for more detailed auditing of creation and update times.

Database Setup

Application Configuration:

Our Spring Boot microservices connect to the MySQL database using standard JDBC settings defined in the application.properties file. For a production environment, these credentials would be stored securely as environment variables.

Data Validation Rules

Input Validation:

- Usernames must be unique and cannot be empty.
- Event names, dates, and locations are required fields.
- Text fields have implicit length limits to ensure data integrity.

Business Rule Checks:

- The application validates that event dates are in the future during event creation and updates.
- The database physically enforces that a user can only have one registration per event.
- Only valid, predefined roles (ROLE_USER, ROLE_ADMIN) can be assigned to users.

Our database design for **BookMySeat** successfully creates a well-organised, scalable, and secure foundation that supports all the event management requirements. The structure efficiently handles the many-to-many relationships between users and events while maintaining data quality through comprehensive application and database rules.

The design supports:

- Multiple user roles with distinct permissions.
- Complex event and booking management.
- Secure, modern authentication and data storage.
- Future growth and the addition of new features.
- Reliable data consistency across the system.

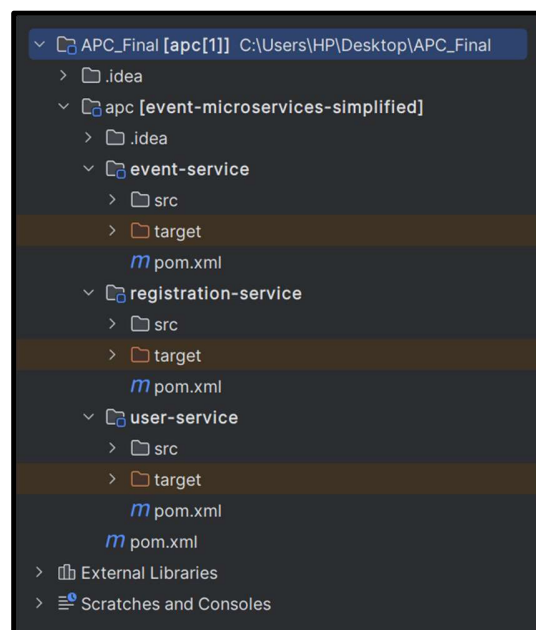
CHAPTER 4: PROGRAM'S STRUCTURE ANALYSING AND GUI CONSTRUCTING (PROJECT SNAPSHOTS)

The BookMySeat system is built on a modern **microservice architecture**, with the application's core logic separated into three independent services. Each service internally follows a classic **three-tier architecture**, consisting of a presentation layer, a business logic layer, and a data access layer. This structured approach ensures maintainability, scalability, and clean code organization throughout the system.

The backend is composed of three distinct Spring Boot microservices: a user-service, event-service, and registration-service. Each service follows a well-organized package structure, with **Controllers** handling web requests and API endpoints, **Services** containing the core business logic, **Repositories** managing data access via Spring Data JPA, and **Entities** defining the database schema.

The authentication system is handled by the user-service and provides secure, password-based login. The entire application, including all inter-service communication, is secured using a stateless **JSON Web Token (JWT)** model, ensuring that every request is properly authenticated without relying on server-side sessions.

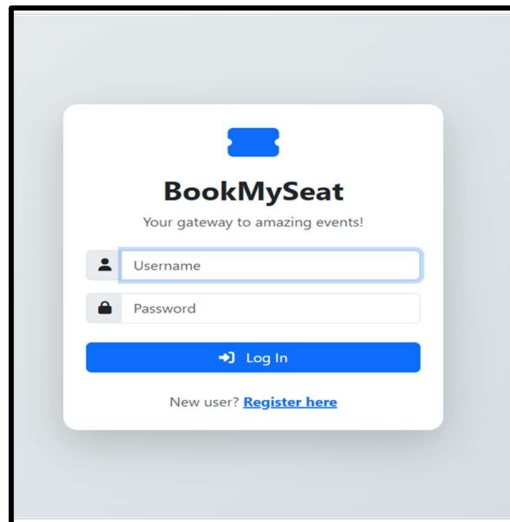
The frontend is built using **Thymeleaf**, a modern server-side Java template engine that integrates seamlessly with Spring Boot. This allows for dynamic web pages to be built on the server. The user interface is styled with the **Bootstrap** framework, ensuring a clean, professional, and **responsive design** that works seamlessly across different devices and screen sizes.



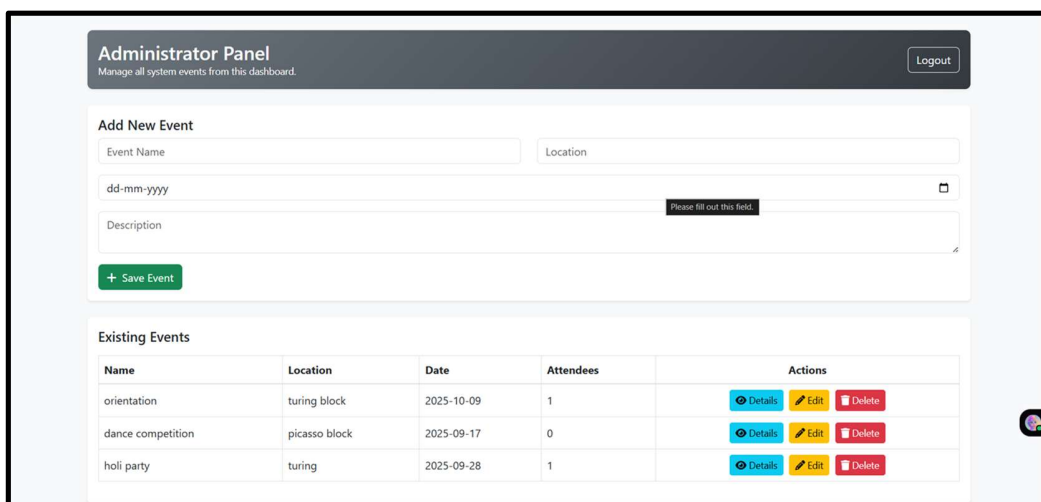
The project directory is structured to reflect the **microservice architecture**, with separate modules for the user-service, event-service, and registration-service. This modular organization is a cornerstone of the project's design, demonstrating a professional approach to software development and a clear **separation of concerns**. Each service operates independently, managing its own specific domain of responsibility, which enhances maintainability and scalability.

The system's user interface is designed with simplicity and usability in mind, featuring intuitive navigation and a clean visual hierarchy. Two distinct interfaces cater to the different user roles:

- The **Admin Dashboard** provides comprehensive management capabilities, allowing administrators to perform full CRUD (Create, Read, Update, Delete) operations on **events** and view detailed lists of registered attendees.
- The **User Dashboard** focuses on providing a seamless experience for browsing available events and managing personal bookings, including registration and cancellation functionalities.



The image shows a login and registration interface for 'BookMySeat'. It features a blue ticket icon at the top, followed by the title 'BookMySeat' and the tagline 'Your gateway to amazing events!'. Below this are two input fields: 'Username' and 'Password', each with a corresponding icon (a person for username and a lock for password). A blue 'Log In' button is positioned below the password field. At the bottom, there is a link for 'New user? Register here'.



The image shows the 'Administrator Panel' interface. It has a dark header with the title 'Administrator Panel' and a 'Logout' button. Below the header is a section for 'Add New Event' with input fields for 'Event Name', 'Location', 'dd-mm-yyyy' (with a calendar icon), and 'Description'. A green '+ Save Event' button is at the bottom of this section. Below the 'Add New Event' section is a table titled 'Existing Events' with columns for 'Name', 'Location', 'Date', 'Attendees', and 'Actions'. The table contains three rows of event data, each with 'Details', 'Edit', and 'Delete' action buttons.

Name	Location	Date	Attendees	Actions
orientation	turing block	2025-10-09	1	Details Edit Delete
dance competition	picasso block	2025-09-17	0	Details Edit Delete
holi party	turing	2025-09-28	1	Details Edit Delete



Update Event

Event Name

Location

Date

Description



BookMySeat

Create your account to join amazing events!

Already registered? [Login here](#)

CHAPTER 5: CONCLUSION

The **BookMySeat** event management system represents a comprehensive solution to the challenges of organizing and attending events. Through the development of this project, we have successfully demonstrated how modern web technologies can transform fragmented, manual processes into an efficient, secure, and user-friendly digital platform.

Our journey in building BookMySeat began with identifying the pain points in conventional event management—inefficient registration tracking, a lack of a central discovery platform for users, and significant administrative overhead. The system we developed addresses these issues through a robust **microservices architecture** that separates concerns while maintaining seamless integration. The use of **Spring Boot** for the backend provided us with a stable and scalable foundation, while **MySQL** offered reliable data storage with strong integrity constraints.

One of the key achievements of this project is the implementation of a modern, **stateless authentication system using JSON Web Tokens (JWT)**. This enhances security and scalability, making the system suitable for a distributed environment. The **role-based access control** system ensures that users and administrators have functionalities tailored to their specific needs, providing a secure and intuitive experience for each role.

The database design, featuring core entities for Users, Events, and Registrations with well-defined relationships, successfully captures the complexity of event booking operations while maintaining data consistency and integrity. The containerized deployment using **Docker** ensures that the application can be reliably deployed across different environments.

Throughout this project, we encountered and overcame numerous technical challenges. Learning to implement JWT authentication across multiple services, designing efficient database relationships for many-to-many connections, managing secure inter-service communication, and ensuring a cohesive user experience from a distributed backend were key hurdles that enhanced our problem-solving skills.

While BookMySeat successfully meets its core objectives, we recognize that there is always room for improvement. The current system provides a solid foundation that can be extended with additional features such as **payment gateway integration** for ticketed events, **email notification systems** for reminders and updates, and advanced event searching and filtering. The modular architecture we've implemented makes these future enhancements feasible without requiring

major restructuring.

This project has been an invaluable learning experience that bridged the gap between academic knowledge and practical application. It has enhanced our skills in software architecture, database design, security implementation, and team collaboration. The successful completion of BookMySeat demonstrates our ability to conceptualize, design, and implement a complex software system that addresses real-world problems.

In conclusion, BookMySeat stands as a testament to how technology can streamline event coordination. The system not only improves operational efficiency for administrators but also enhances the overall experience for users seeking to engage with events. As we move forward, the skills and knowledge gained from this project will undoubtedly serve as a strong foundation for our future endeavours in software development and system design.

CHAPTER 6: FUTURE SCOPE

The BookMySeat system, while comprehensive in its current form, presents numerous opportunities for enhancement and expansion. The modular microservice architecture and scalable design provide a solid foundation for implementing advanced features that can further enrich the event management experience. Several exciting developments can be envisioned to build upon the existing system.

One of the most significant enhancements would be the integration of a **payment gateway** such as Stripe or Razorpay. This would allow administrators to host paid events, manage different ticket tiers, process secure online payments, and handle refunds, transforming the platform into a complete ticketing solution.

The development of a **dedicated mobile application** represents a natural progression for BookMySeat. While the current web application is responsive, a native mobile app would provide better performance, **push notifications** for important updates (like event reminders or cancellations), and seamless integration with device features. The mobile app could include features such as digital tickets with QR codes for easy check-in and offline access to event details.

Advanced analytics and reporting capabilities could transform how organizers plan their events. By analyzing registration data, the system could provide insights into event popularity, peak booking times, and attendee demographics. Interactive dashboards with these trends would provide valuable information for administrative decision-making.

Integration with third-party systems would create a more cohesive ecosystem. BookMySeat could connect with social media platforms for easy event sharing and with calendar services like Google Calendar or iCal, allowing users to add events to their personal schedules with a single click. **Single Sign-On (SSO)** integration would streamline the authentication process across different platforms.

Enhanced features could include **customizable event landing pages**, image and video galleries for events, and a user review and rating system. An **automated check-in system** using QR codes on digital tickets could streamline entry at the venue while providing organizers

with real-time attendance data.

Accessibility improvements would ensure the system serves all users equally. This could include features for visually impaired users by following WCAG (Web Content Accessibility Guidelines) and multi-language support to make the system more inclusive for international users.

In conclusion, the future of BookMySeat is bright with numerous possibilities for growth and innovation. Each enhancement would build upon the solid foundation already established, creating an increasingly sophisticated and comprehensive event management ecosystem. The system's modular architecture ensures that these future developments can be implemented incrementally, allowing for continuous improvement while maintaining system stability and reliability.

CHAPTER 7: BIBLIOGRAPHY/REFERENCES

- <https://stackoverflow.com/questions>
- <https://www.youtube.com/watch?v=xrj3zzaqODw>
- <https://www.docker.com/>